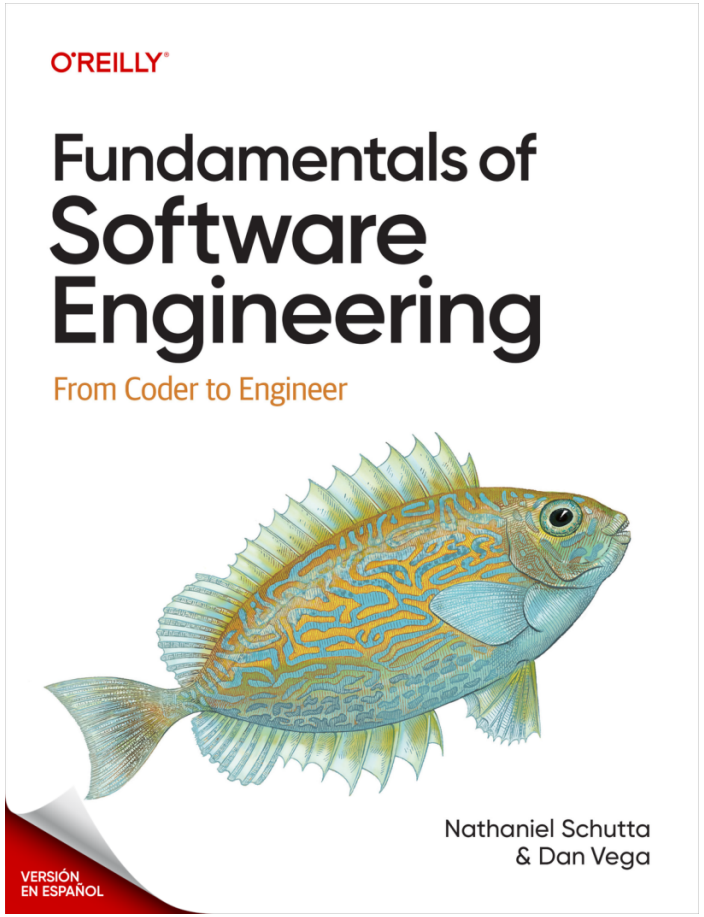
Tecnológico de Software

Aprender ingeniería en la era de la IA

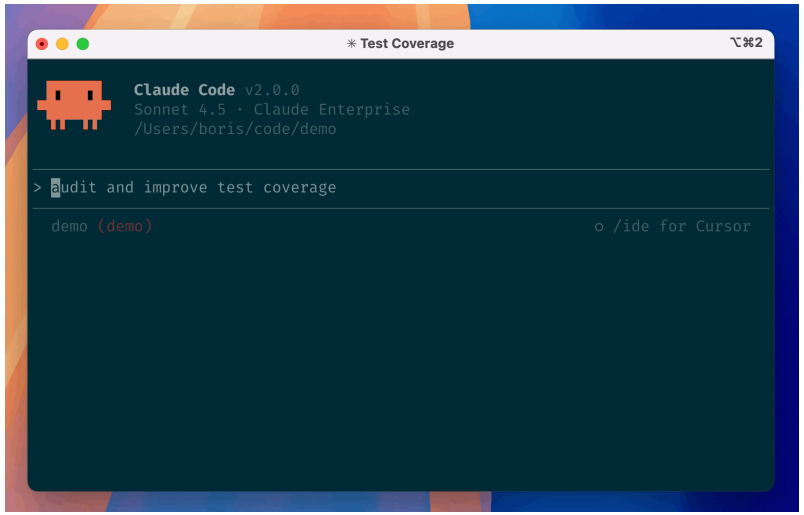


Lo que me dejaron los primeros cinco capítulos

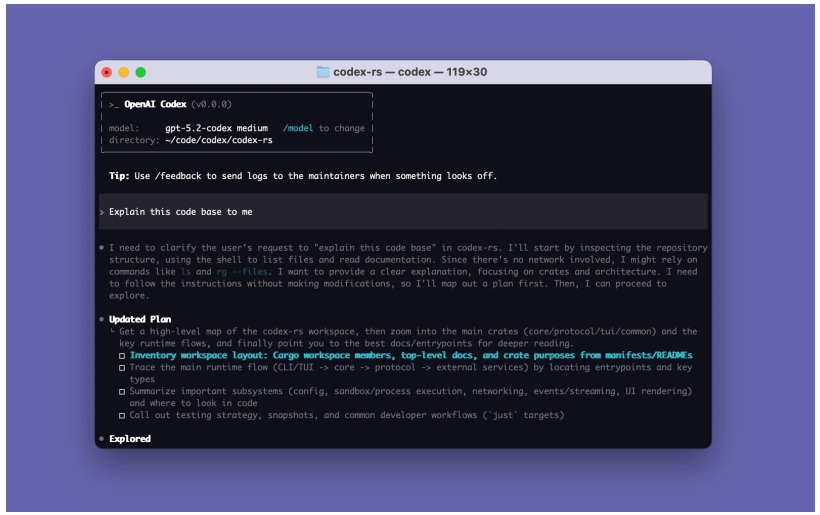
Fernando
Programador junior

La pregunta que me trajo al libro

¿Cómo desarrollo criterio propio cuando puedo delegar tanto código?



Claude Code



Codex

Los nombres y el alcance del trabajo

Programador o codeador

Su foco suele estar en implementar código.

Desarrollador

Comprende mejor la entrega del producto y su contexto.

Ingeniero de software

Asume decisiones sobre todo el ciclo de vida del sistema.

El libro propone una distinción de alcance. En la práctica, los títulos se solapan.



El aprendizaje continúa en sistemas reales

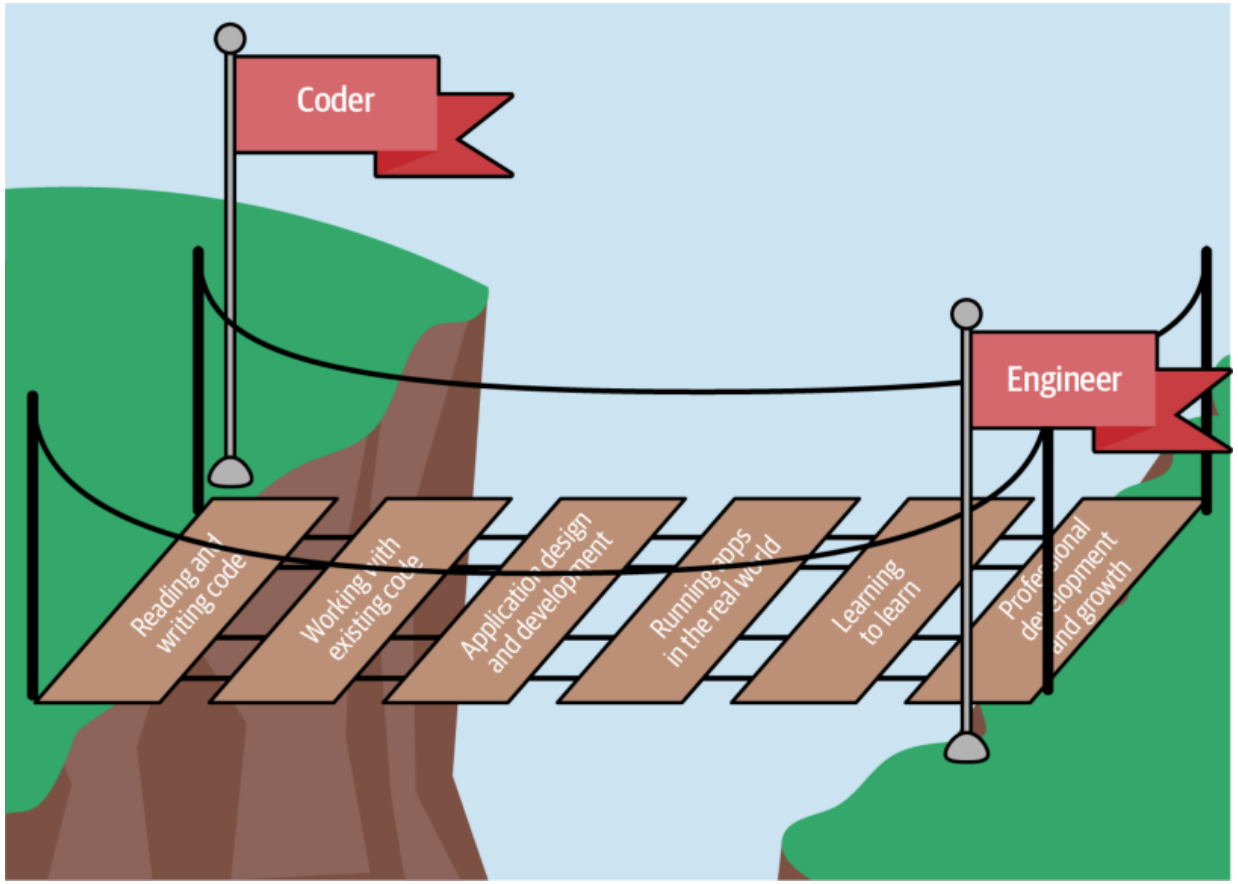


Figura P-1. La brecha entre aprender a programar y ejercer ingeniería

El código vive mucho tiempo en mantenimiento.

Aprendo al entender decisiones ajenas y mejorar lo que existe.

Mi primera tarea es entender el problema

Antes de pedir una implementación…

¿Qué necesita la persona?

El objetivo empieza en su problema.

¿Qué restricciones importan?

Tiempo, riesgos y contexto del sistema.

¿Cómo sabremos que funciona?

Un resultado observable y comprobable.



Leemos más código del que escribimos

Leer es reconstruir la intención de otra persona.

Antes de juzgar una solución, necesito conocer su contexto.

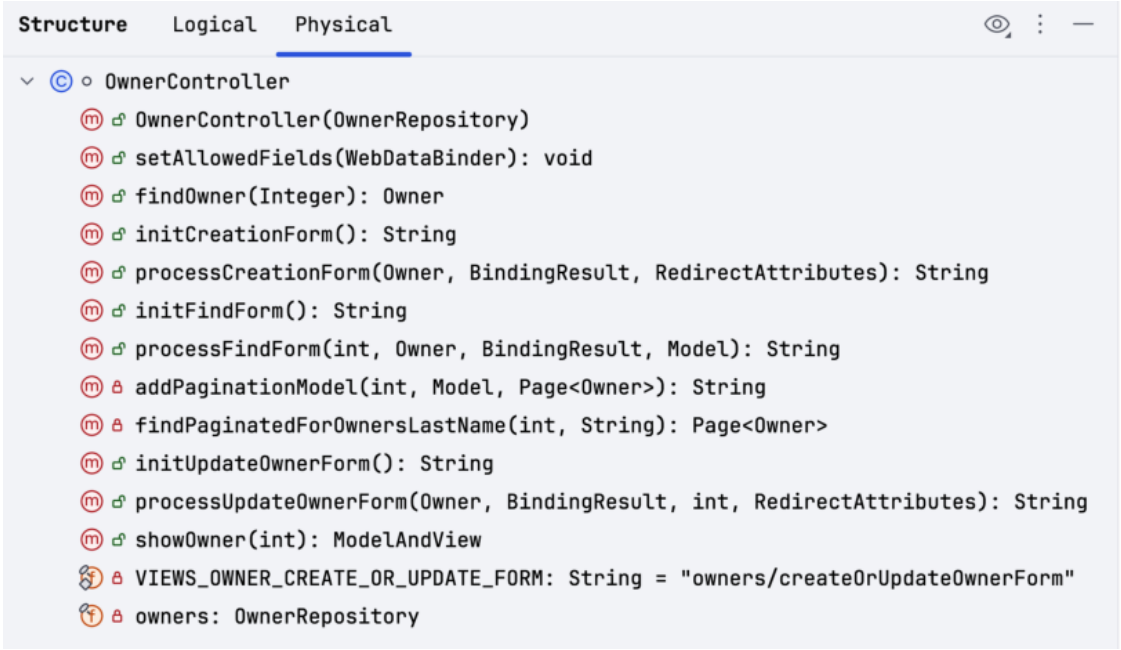


Figura 2-12. Vista de estructura de una clase



Una ruta para explorar código desconocido

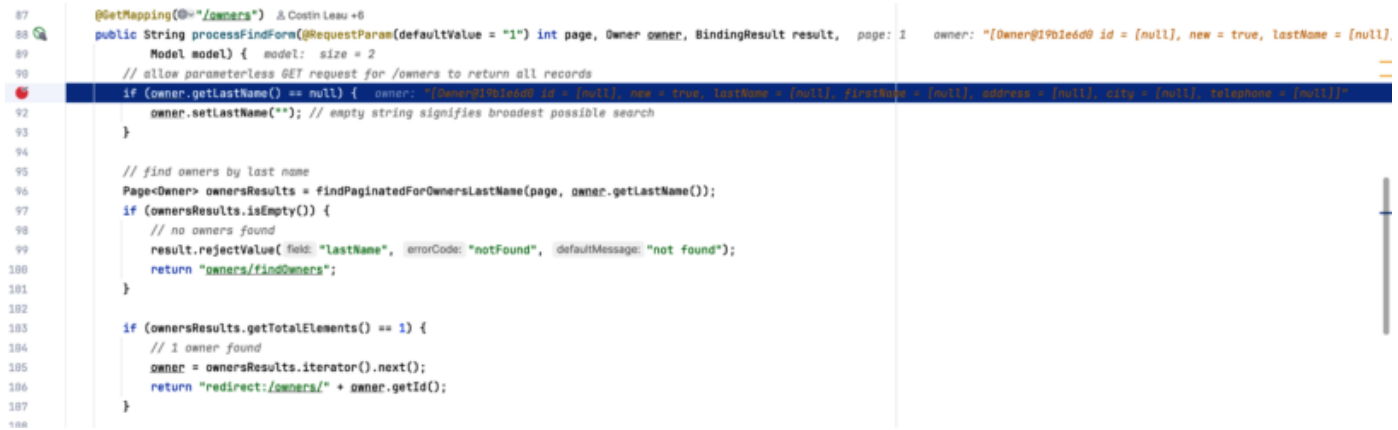


Figura 2-2. Inspección del estado con un punto de interrupción

IntelliJ IDEA

1. Ejecutar

Elegir un caso de uso.

2. Seguir

Leer llamadas y pruebas.

3. Contrastar

Depurar y revisar el historial.

El código también lo leen personas

Intención escondida

```
s = sum(x.v for x in xs
        if x.ok)
```

Intención visible

```
total_pagado = sum(
    pago.importe
    for pago in pagos
    if pago.confirmado
)
```

La estructura y los nombres deben ayudar a comprender el comportamiento.



Buenas prácticas para mantener software

Responsabilidades claras

Una función debe poder explicarse con facilidad.

Complejidad justificada

La abstracción necesita un problema real.

Convenciones compartidas

Nombres del dominio y decisiones consistentes.

Revisión entre personas

Preguntar, argumentar y aceptar retroalimentación.



La documentación conserva el contexto

Lo que otra persona necesita

Cómo ejecutar el proyecto
Por qué tomamos una decisión
Qué comportamiento esperamos
Qué cambió y dónde impacta

Se actualiza junto con el código.

Ejemplo de una decisión

Contexto
Un pago puede reenviarse.

Decisión
Identificar cada operación.

Motivo
Evitar cobros duplicados.



Modelar ayuda a conversar antes de construir

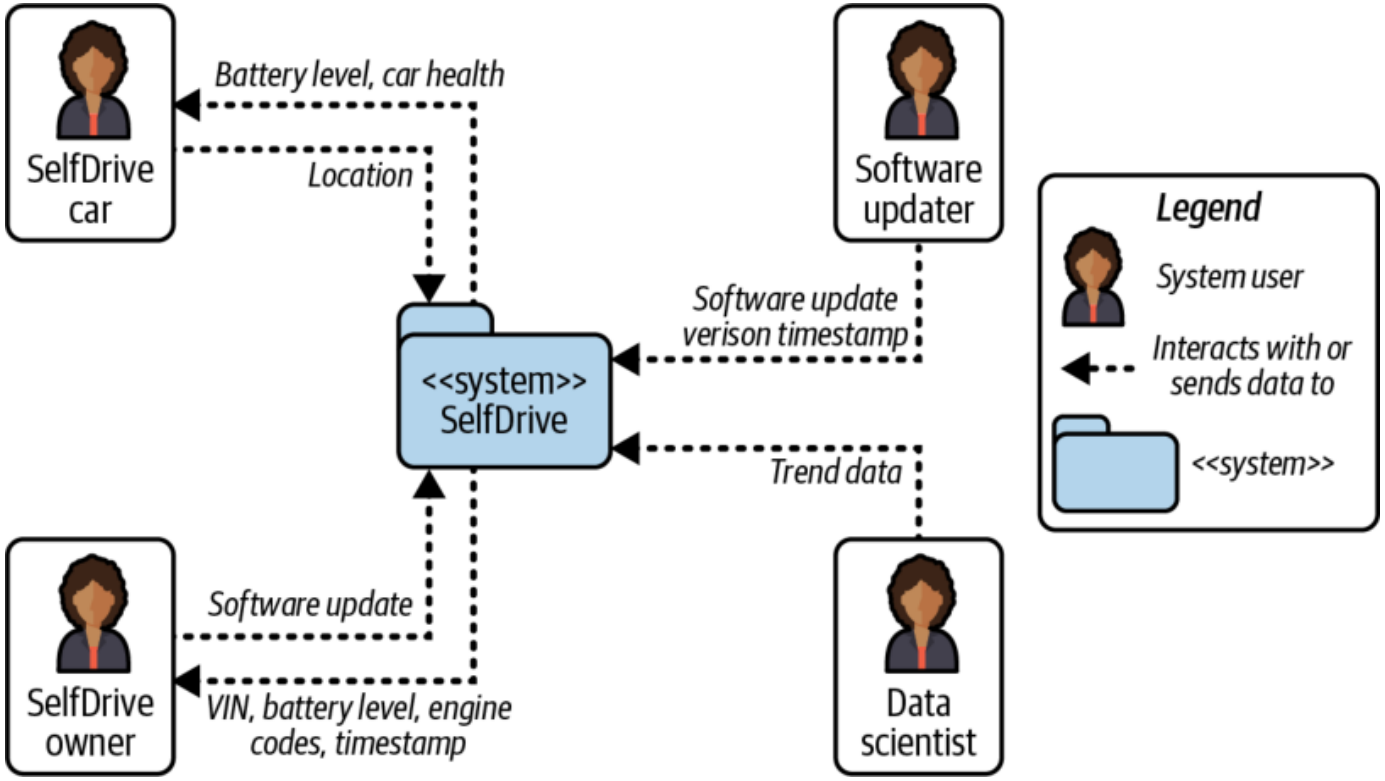


Figura 4-2. Diagrama de contexto del libro

¿Quién usa el sistema y qué intercambia con él?

El modelo hace visibles los límites y los supuestos.

Cada modelo responde una pregunta

Contexto

¿Quién se relaciona con el sistema?

Componentes

¿Cómo se reparten las responsabilidades?

Secuencia

¿En qué orden colaboran las partes?

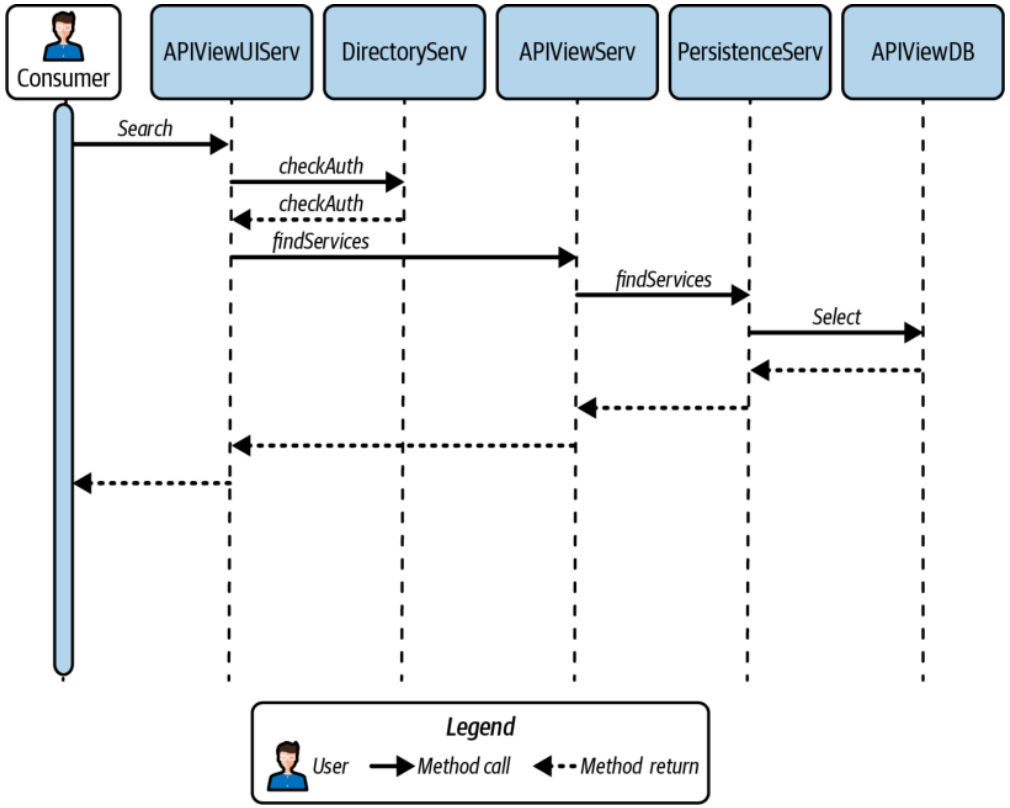


Figura 4-5. Ejemplo de diagrama de secuencia

La sencillez depende del público

Un modelo es útil cuando su audiencia puede explicar lo importante.

Para conversar con negocio

Actores, objetivos y límites.
Vocabulario del problema.

Para conversar con el equipo

Responsabilidades e interacciones.
Detalle técnico suficiente.



Las pruebas me dan confianza para cambiar

Puedo refactorizar con más confianza cuando tengo evidencia del comportamiento.

01

Antes

Comprobar el estado actual.

02

Durante

Cambiar una responsabilidad.

03

Después

Buscar regresiones.



La pirámide orienta la combinación de pruebas

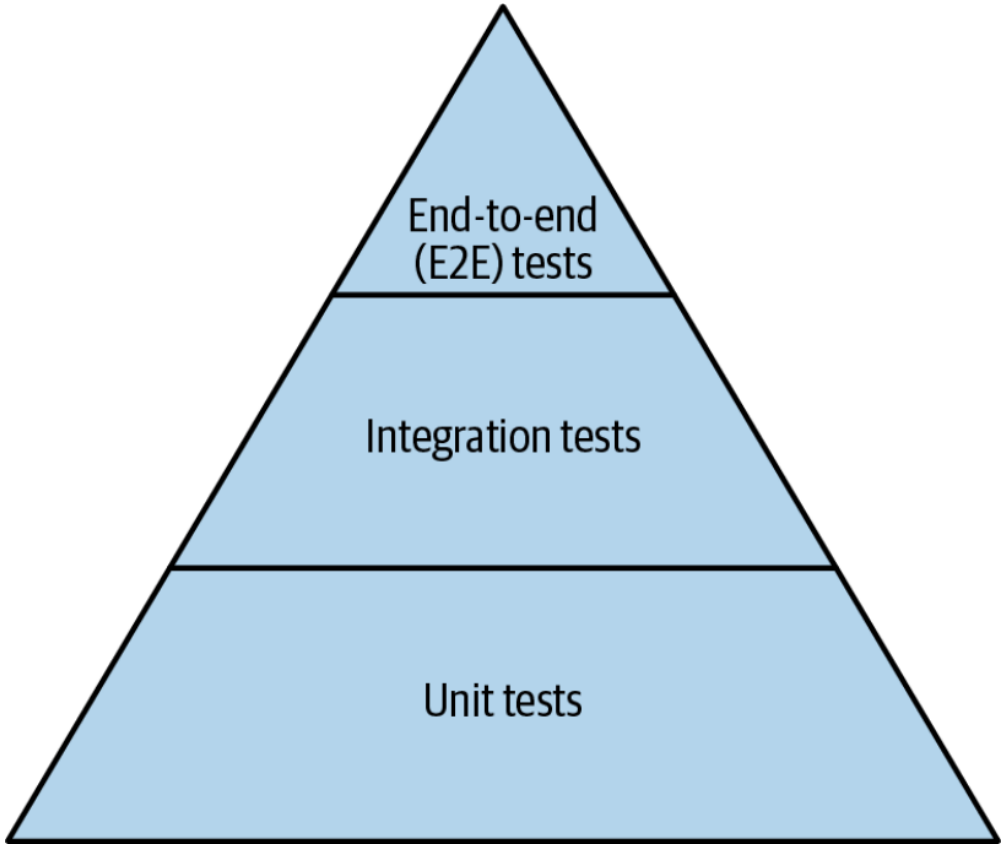


Figura 5-1. Pirámide de pruebas del libro

Extremo a extremo

Un recorrido completo del usuario.

Integración

La colaboración entre componentes.

Unitarias

Una regla o función aislada.



Una prueba debe comprobar algo importante

Ejecutar código

```
cobrar(pedido)
```

Verificar comportamiento

```
resultado = cobrar(pedido)
comprobar(resultado.total, 100)
comprobar(cobros_creados, 1)
```

La cobertura señala zonas por revisar.
La aserción comprueba lo que esperamos.



Así quiero trabajar con agentes de IA

Antes de delegar

Entender el problema y definir qué aceptaré.

Durante el cambio

Pedir un alcance pequeño y leer la propuesta.

Antes de integrar

Explicar el diff, probar y actualizar el contexto.

La responsabilidad sobre el resultado sigue siendo mía.



El aprendizaje que quiero conservar

La práctica que forma criterio

Leer código ajeno
Investigar un fallo
Recibir una revisión
Mantener una decisión

Mi práctica con IA

Formular mi hipótesis primero
Comprobar las explicaciones
Defender mis decisiones
Revisar el efecto del cambio

Mi siguiente paso como junior

Quiero poder explicar y sostener lo que construyo.

En mi próximo cambio voy a dejar:

Una explicación clara del comportamiento.
Una prueba que proteja lo importante.
Contexto útil para la siguiente persona.



Lecturas e imágenes

Nathaniel Schutta y Dan Vega

Fundamentos de la ingeniería de software
O’Reilly, 2026. Prefacio y capítulos 1 al 5.

Figuras del libro

P-1, 2-2, 2-12, 4-2, 4-5 y 5-1.
Edición suministrada por el expositor.

Imágenes de herramientas

Repositorios oficiales de Anthropic, OpenAI y JetBrains. Sitio de JUnit.
Enlaces completos en las notas.

